

Part 4 — PS Feature 3: Dynamic Pricing Assistant

The slide claims an ML model. The code is arithmetic. **The arithmetic is the better answer** — this section is how to say so without it sounding like a retreat.

Q4.1 — "Your slide says a machine learning algorithm suggests the price. Walk me through the model. What did you train it on?"

Think. There is no model. Correct the slide in my own words before they find it, and make the correction the argument.

Answer. There is no model, the slide overstates it, and that is deliberate — here is why.

No training data exists for "what should this handicraft cost." Not "we could not find it" — it does not exist. Every handloom piece is a different weave, count, dye, size and hand. A price predictor with no training set is a slide we cannot defend under one question, and on a government problem statement **every number has to survive "how did you get that?"**

So: **pricing is arithmetic, not a model** (docs/decisions.md). Four layers, and they map exactly onto the PS's own wording, which names "raw material costs" as an input:

```
① material_cost          artisan-stated, voice question q_cost
② labour_hours × wage_rate  rates.json, per cluster
=====
cost = ① + ②
③ × (1 + default_margin_pct)
=====
      FLOOR          ← never go below this
④ market comparables    mid = (low + high) / 2
  price = max(floor, mid)  ← the market can only move us UP
  below_floor_warning = high < floor
⑤ mrp = price / (1 - discount)  ← GeM mandates ~10% off MRP
```

Current rates (ai/price/rates.json): default wage ₹120/hour, default margin 15%, cluster rates for **sambalpur 120, bhuj 140, varanasi 150, channapatna 110**, and a per-channel minimum discount of **10% for GeM**, zero elsewhere.

There is now no LLM in this feature at all. An earlier version of this answer said the model appeared exactly once, in `comps.normalize()`, to tidy messy comparable titles. **That function was deleted on 2026-09-03** — it was never called, and the reason given for removing it is the sharper version of our own rule: "an LLM normalising listing titles would put a model in the one place F3 says no model goes."

So the boundary in `ai/README.md` — **no LLM touches a price** — is now enforced by there being no model in the path, rather than by a convention someone has to remember.

[warning] **One stale docstring to fix before anyone reads the source in the room:** `ai/price/comps.py` still opens with *"then uses an LLM to normalize messy listing titles."* The function is gone; the sentence is not. A judge who opens that file sees us describing something we deliberately removed.

The correct slide sentence: *"Deterministic cost-up pricing with a loss-guard floor. Every rupee traces to a material cost the artisan stated, hours they worked, and a published cluster wage rate."*

Breaks when. A judge wants ML because the PS says "dynamic." Answer: dynamic means it responds to inputs that change — material cost, hours, cluster rate, channel discount, comparables. It does. What it does not do is guess.

Q4.2 — "Then what is the AI in your AI pricing assistant? Be honest."

Think. Do not stretch. Name the one part that is genuinely intelligent and let the rest be arithmetic proudly.

Answer. Almost none, deliberately — and that is the point.

The intelligence sits **upstream** of the price, not inside it:

1. **The voice interview** that extracts material, hours and cost from ordinary spoken sentences — including a dialect *"ee sutti ke saari ha"* that a naive prompt got confidently wrong (Q3.8). Getting `labour_hours` out of *"do din laga"* is the hard part; multiplying it by a wage rate is not.
2. **Category mapping**, which is where the real intelligence in this whole system lives — see Q5.2. **The price is only as good as the category it is compared within.**

Inside the price calculation itself there is now no model whatsoever. Say that plainly. On a government problem statement, *"the arithmetic is auditable and no model can move it"* is a feature, not an apology.

And the honest framing that wins the exchange: the intelligence in this feature is not in the algorithm, it is in **knowing which number is wrong.** Every pricing tool in the market optimises the price upward. We built a floor, because in this sector the error is in the other direction.

Q4.3 — "What is the floor actually protecting them from?"

Think. This is the question where the feature is strongest. Answer with the scenario, not the formula.

The problem is not that artisans do not know the market rate. It is that they routinely price **below what the thing cost them to make**, because **their own labour is the one input they were never taught to count.**

A middleman offers ₹1,800 for eleven days of work and ₹800 of thread, and it gets accepted — because ₹1,800 feels like money and the eleven days feel like Tuesday.

`docs/Master-Technical-Reference.md` §7.2 ④ puts it as: **under-pricing is the actual epidemic in this sector,**

not over-pricing. This single feature does more for the PS's "increase average annual income" impact goal than anything else we built.

Which is why, in `docs/app/Pricing.md` §0:

The price floor is the ethical core of this feature, not a validation rule.

*It is not there to keep the data clean. It is there because a system that helps an artisan list a product faster, and lets them list it at a loss, **has made their life worse with better UX.***

Two consequences in the code:

- **The floor warning is always spoken, never silent**, and spoken again every time they try to go under it (`/price`, route 14). A silent validation message is useless to this user.
- `/price` refuses with 422 when there is neither a material cost nor labour hours — deliberately, because a floor of ₹0 clamps nothing while sounding authoritative. Answering with a fake floor would be worse than answering with nothing.

Market comparables can only move the price UP (`price = max(floor, mid)`). If the market will not pay what it cost, we do not quietly lower them to market — we set `below_floor_warning` and tell them.

Q4.4 — "The artisan tells you it took eleven days. They are guessing, or they are exaggerating. Your entire floor rests on a self-reported number."

Think. This is the sharpest attack on this feature and it is correct. Concede it precisely, then show it is bounded.

Answer. Correct, and our own research says so before you do. `research/RESULTS.md`, the pricing row, verdict 2026-08-28: "first evidence — depends entirely on labour hours." We recorded the weakness as the finding.

Why it is still the right design. The alternatives are worse:

- A market-down price ignores their labour entirely — which is the exact failure we are correcting.
- An inferred labour estimate would be **our** guess about **their** work, presented with false authority. If we are going to be wrong, the artisan should be the one who was wrong, about their own hours, with the arithmetic visible.

What bounds it:

- **The breakdown is spoken, itemised.** "Dhaaga ₹800, aapka kaam 40 ghante × ₹120 = ₹4,800." An artisan who hears their own hours read back catches an exaggeration in a way they never would reading a total.
- **cost_material and labour_hours are persisted on the product**, not just handed to `/price` — so a listing can be re-priced later when a rate changes, and so the stated hours are auditable against what the same artisan says for a similar piece.
- **Cluster wage rates are ours, not theirs.** They cannot inflate ₹120/hour.

- **Exaggeration is self-correcting through the market.** An inflated floor produces a price that does not sell. Under-reporting is the dangerous direction, and under-reporting is the behaviour we are actively fighting.

At scale. Once we have order data, the honest check is distributional: hours-per-piece by craft and cluster, flagging outliers for the coordinator rather than overriding the artisan. **We should never silently replace a stated number with a modelled one on a screen about their own income.**

Breaks when. A first-time artisan with no reference for their own hours, pricing their first piece. Nothing in the system helps them. A per-craft median hours hint — *"is jaisi saree me aam taur par 35–45 ghante lagte hain"* — is the fix, and it needs data we do not have yet.

Q4.5 — "Show me your market comparables. Where do they come from?"

Think. They are thin. Be exact about how thin, because the collection *rule* is the impressive part.

Today. `ai/price/comps.py` — the live comparables lookup — **returns None**, and `comps.py:147` is a `NotImplementedError`. What exists is `ai/price/comps_seed.json`: **144 listings hand-collected from indiahandmade.com** on 2026-08-28, covering **three** category keys.

Three, not thirty. And the biggest weakness is named in `research/pricing/README.md`: we need weave-specific keys — `textiles.saree.sambalpuri`, not `textiles.saree` — because a Sambalpuri bandha and a printed powerloom saree are not the same comparison class, and averaging them produces a number that is wrong for both.

Why collected by hand, and why that is not laziness:

Source	Why there is no API
GeM	No seller or catalogue API of any kind. Rate contracts are published as documents
Amazon / Flipkart	Seller APIs. They authenticate as one shop and return that shop's own listings. There is no open "what does a cotton saree go for" endpoint — and our artisans have no seller account to authenticate with in the first place
indiahandmade	Public catalogue, browsable without an account. The best source we have

indiahandmade is the right comparison class, not an approximation — the Ministry of Textiles' own D2C marketplace for verified weavers and GI-tagged products. Same artisans, same crafts, same handmade claim. On a government PS, nobody has to be persuaded of that.

A scraper was considered and rejected: against both sites' terms, breaks the week somebody renames a CSS class, and **gets the demo laptop IP-blocked halfway through a presentation**. An afternoon of browsing produces better data with none of that risk, and *"a dated sample of real listings"* is a true sentence in the room.

The one rule in that folder, and it is enforced in code:

Never invent a number here. `pricing.py` refuses any row with no `seen_on` date and no `url_or_note`. Provenance is the entire difference between research and a number somebody felt was about right. A plausible-looking guess is worse than an empty file: an empty file prices honestly on cost alone and the app says so; a guess does not.

At scale. Our own marketplace generates real transaction data — that is one of the six reasons it exists (`docs/Master-Technical-Reference.md` §3.3). Every sale is a true comparable in exactly our category taxonomy, unlike anything scraped. Comparables improve as a function of usage, which is the only version of "self-improving" in this system that is actually true.

Breaks when. A craft outside the three seeded keys. `comps` returns nothing, `price = floor`, and the app says so rather than inventing a market. That is the designed behaviour and it is correct — but it means for most crafts today, "competitive market price" is not something we supply.

Q4.6 — "GeM forces a discount off MRP. Does your price survive it?"

Think. This is a specific, checkable, easily-missed detail. Getting it right is worth more than it looks, because it proves we read the platform rules rather than the platform marketing.

Today. Yes, and it is layer ⑤. GeM **mandates a minimum discount off MRP** when listing, ~10% typical. `rates.json` carries `channel_min_discount_pct`: `{"gem": 0.1, ...}`.

The trap: if we suggest ₹2,000 and GeM knocks 10% off, the artisan nets ₹1,800 — **which may be below their floor**. A pricing engine that runs the floor guard *before* the discount recommends loss-making prices on the platform we headlined.

So the floor guard runs AFTER the mandated discount, and MRP is set so the post-discount price still clears the floor: `mrp = price / (1 - discount)`.

The gap I would flag before being asked. `app/src/screens/Price.jsx:84` **hardcodes** `channel: 'gem'`. That is deliberate — GeM has the strictest discount rule, so pricing for GeM produces an MRP that is safe everywhere else. But it means the artisan sees one price, not a per-channel price, and if they publish only to Amazon they are carrying a GeM-shaped MRP for no reason. Per-channel pricing is a UI decision we have not made, not a missing calculation.

Q4.7 — "Does your floor land anywhere near what these things actually sell for?"

Think. We have a first verdict and it is qualified. Give it with the qualification attached.

Today. `research/RESULTS.md`, pricing row, 2026-08-28: **"first evidence — depends entirely on labour hours."**

That is the whole verdict. It is not "our floor matches the market"; it is "the answer is dominated by one self-reported input."

Which is arithmetically obvious once stated — at ₹120/hour, the difference between 20 hours and 60 hours is ₹4,800, and on a saree in the ₹2,000–₹6,000 band that swamps the material cost entirely. **Labour is not a term in this formula. It is the formula.**

What that tells us to build next, in order: 1. Per-craft hours priors, so an artisan has a reference for their own time. 2. Weave-specific comparable keys, so the market side stops averaging incomparable things. 3. Real transaction data from our own marketplace.

Breaks when. A judge asks for a worked example against a real listing. We can do exactly one category family credibly — the seeded ones — and we should offer that one rather than a general claim.